

Chapter 7: Nullability

By Irina Galata

All the variables and constants you've dealt with so far have had concrete values. When you had a string variable, like `var name`, it had a string value associated with it, like "Joe Howard". It could have been an empty string, like `""`, but nevertheless, there was a value to which you could refer.

That's one of the built-in safety features of Kotlin: If the type says `Int` or `String`, then there's an actual integer or string there — guaranteed.

This chapter will introduce you to **nullable** types, which allow you to represent not just a value, but also the absence of a value. By the end of this chapter, you'll know why you need nullable types and how to use them safely.

Introducing null

Sometimes, it's useful to be able to represent the absence of a value. Imagine a scenario where you need to refer to a person's identifying information; you want to store the person's name, age and occupation. Name and age are both things that must have a value — everyone has them. But not everyone is employed, so the absence of a value for occupation is something you need to be able to handle.

Without knowing about nullables, this is how you might represent the person's name, age and occupation:

```
var name = "Joe Howard"  
var age = 24  
var occupation = "Software Developer & Author"
```

But what if I become unemployed? Maybe I've reached enlightenment and wish to live out the rest of my days on top of a mountain. This is when it would be useful to be able to refer to the absence of a value.

Why couldn't you just use an empty string? You *could*, but nullable types are a much better solution. Read on to see why.

Sentinel values

A valid value that represents a special condition, such as the absence of a value, is known as a **sentinel value**. That's what your empty string would be in the previous example.

Let's look at another example. Say your code requests something from a server, and you use a variable to store any returned error code:

```
var errorCode = 0
```

In the success case, you represent the lack of an error with a zero. That means 0 is a sentinel value. Just like the empty string for occupation, this works, but it's potentially confusing for the programmer. 0 might actually be a valid error code — or could be in the future, if the server changed how it responded. Either way, you can't be completely confident that the server didn't return an error without consulting the documentation. In these two examples, it would be much better if there were a special type that could represent the absence of a value. It would then be explicit when a value exists and when one doesn't.

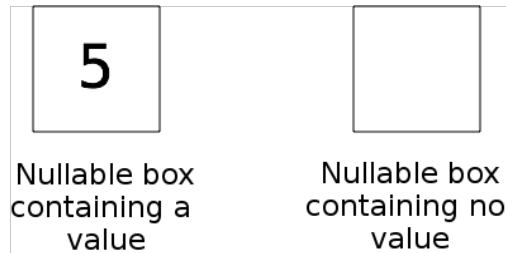
Null is the name given to the absence of a value, and you're about to see how Kotlin incorporates this concept directly into the language in a rather elegant way. Some other programming languages simply use sentinel values. Some have the concept of a null value, but it is merely a synonym for zero. It is just another sentinel value.

Kotlin introduces a whole new set of types, **nullable types**, that handles the possibility a value could be `null`. If you're handling a non-null type, then you're guaranteed to have a value and don't need to worry about the existence of a valid value. Similarly, if you are using a nullable type, then you know you must handle the `null` case. It removes the ambiguity introduced by using sentinel values.

Introducing nullable types

Nullables are Kotlin's solution to the problem of representing both a value and the absence of a value. A nullable is allowed to hold either a value *or* `null`.

Think of a nullable as a box: it either contains a value, or it doesn't. When it doesn't contain a value, it's said to contain `null`. The box itself always exists; it's always there for you to open and look inside.



A `String` or an `Int`, on the other hand, doesn't have this box around it. Instead there's always a value, such as `"hello"` or `42`. Remember, non-null types are guaranteed to have an actual value.

Note: Those of you who've studied physics may be thinking about Schroedinger's cat right now. Nullables are a little bit like that, except it's not a matter of life and death!

You declare a variable of a nullable type by using the following syntax:

```
var errorCode: Int?
```

The only difference between this and a standard declaration is the question mark at the end of the type. In this case, `errorCode` is a “nullable `Int`”. This means the variable itself is like a box containing either an `Int` or `null`.

Note: You can add a question mark after any type to create a nullable type. For example, nullable type `String?` is a nullable `String`. In other words: a nullable box of type `String?` holds either a `String` or `null`.

Also, note how a nullable type must be made explicit using a type declaration (here `: Int?`). Nullable types can never be inferred from initialization values, as those values are of a regular, non-null type.

The exception is if you’re assigning the value using the result of a function call. If the return type of the function is nullable, the variable will be inferred to be nullable.

Setting the value is simple. You can either set it to an `Int`, like so:

```
errorCode = 100
```

Or you can set it to `null`, like so:

```
errorCode = null
```

This diagram may help you visualize what’s happening:

`errorCode` = 

`errorCode` = 

The nullable box always exists. When you assign `100` to the variable, you’re filling the box with the value. When you assign `null` to the variable, you’re emptying the box.

Take a few minutes to think about this concept. The box analogy will be a big help as you go through the rest of the chapter and begin to use nullables.

Mini-exercises

1. Make a nullable `String` called `myFavoriteSong`. If you have a favorite song, set it to a string representing that song. If you have more than one favorite song or no favorite, set the nullable to `null`.
2. Create a constant called `parsedInt` and set it equal to `"10".toIntOrNull()`; this will try to parse the string `"10"` and convert it to an `Int`. Check the type of `parsedInt` by clicking `toIntOrNull()` and holding `Control + Shift + P`. Why is it a nullable?
3. Change the string being parsed in the above exercise to a non-integer (try `dog` for example). What does `parsedInt` equal now?

Checking for null

It's all well and good that nullables exist, but you may be wondering how you can look inside the box and manipulate the value it contains.

In some limited cases, you can just use the nullable as if it were a non-null type.

Take a look at what happens when you print out the value of a nullable:

```
var result: Int? = 30
println(result)
```

This just prints out 30.

To see how a nullable type is different from a non-null type, see what happens if you try to use `result` in an expression as if it were a normal integer:

```
println(result + 1)
```

This code triggers an error:

```
Operator call corresponds to a dot-qualified call
'result.plus(1)' which is not allowed on a nullable receiver
'result'.
```

It doesn't work because you're trying to add an integer to a box — not to the value inside the box, but to the box itself. That doesn't make sense!

Not-null assertion operator

The error message gives an indication of the solution: It tells you that the nullable is still inside its box. You need to remove the value from its box. It's like Christmas!

Let's see how that works. Consider the following declarations:

```
var authorName: String? = "Joe Howard"
var authorAge: Int? = 24
```

There are two different methods you can use to remove these nullables from the box. The first is using the **not-null assertion operator** `!!`, which you do like so:

```
val ageAfterBirthday = authorAge!! + 1
println("After their next birthday, author will be
$ageAfterBirthday")
```

This code prints:

```
After their next birthday, author will be 25
```

Great! That's what you'd expect.

The double-exclamation mark after the variable name tells the compiler that you want to look inside the box and take out the value. The result is a value of the non-null type. This means `ageAfterBirthday` is of type `Int`, not `Int?`.

The use of the word “assertion” and the exclamation marks `!!` probably conveys a sense of danger to you, and it should. You should use not-null assertions sparingly. To see why, consider what happens when the nullable doesn't contain a value:

```
authorAge = null
println("After two birthdays, author will be ${authorAge!! +
2}")
```

This code produces the following runtime error:

```
Exception in thread "main" kotlin.KotlinNullPointerException
```

The **null-pointer exception** occurs because the variable contains no value when you try to use it. What's worse is that you get this exception at runtime rather than compile time — which means you'd only notice the exception if you happened to execute this code with some invalid input. Worse yet, if this code were inside an app, the null-pointer exception would cause the app to crash!

How can you play it safe? For that, you'll turn to the second way to get a value out of the nullable.

Smart casts

Under certain conditions, you can check whether a nullable has a value, and if so, you can use the variable as if it were not `null`:

```
var nonNullableAuthor: String
var nullableAuthor: String?

if (authorName != null) {
    nonNullableAuthor = authorName
} else {
    nullableAuthor = authorName
}
```

You'll immediately notice that there are no exclamation marks here when using the nullable `authorName`. Using nullable checks in this way is an example of Kotlin **smart casts**.

If the nullable contains a value, the `if` expression then executes the first block of code, within which Kotlin will smart cast `authorName` to a regular non-null `String`. If the nullable doesn't contain a value, then the `if` expression executes the `else` block.

You can see how using smart casts is much safer than not-null assertions, and you should use them whenever a nullable might be `null`. Not-null assertion is only appropriate when a nullable *is guaranteed* to contain a value.

Using smart casts for nullables is only helpful if the nullable being checked is not or cannot be changed after the null check occurs. For example, if the nullable is assigned to a `var` that is not-changed after the smart cast occurs and before usage or is assigned to a `val`.

Now you know how to safely look inside a nullable and extract its value, if one exists.

Mini-exercises

1. Using your `myFavoriteSong` variable from earlier, use a null check and smart cast to check if it contains a value. If it does, print out the value. If it doesn't, print "I don't have a favorite song."
2. Change `myFavoriteSong` to the opposite of what it is now. If it's `null`, set it to a string; if it's a string, set it to `null`. Observe how your printed result changes.

Safe calls

Suppose you want to do something with a nullable string other than print it, such as accessing its length. Using a smart cast inside a null check is overkill for such a simple use of the string. If you try to access the length as if the string were not-nullable and without a smart cast, you'll get a compiler error:

```
Only safe (?.) or non-null asserted (!!) calls are allowed on a nullable receiver of type String?
```

The error tips you off to the solution, which is to use a **safe call** with the `?.` operator:

```
var nameLength = authorName?.length
println("Author's name has length $nameLength.")
// > Author's name has length 10.
```

By using the safe call operator, you're able to access the `length` property.

Safe calls can be chained:

```
val nameLengthPlus5 = authorName?.length?.plus(5)
println("Author's name length plus 5 is $nameLengthPlus5.")
// > Author's name length plus 5 is 15.
```

If a safe call is made on a value that is null, the expression stops evaluating the chain and returns null.

Since the result of a safe call can be null, expressions using safe calls on nullables return nullable types. For example, `nameLength` above is of type `Int?` and not `Int`, even though the `length` property on a string is not-nullable. The type of the entire expression is nullable.

The `let()` function

The safe call operator provides another way to use smart casts to work with the non-null value inside a nullable, via the `let()` function from the standard library:

```
authorName?.let {
    nonNullableAuthor = authorName
}
```


Within a `let` function call following the safe call operator, the variable becomes non-nullable, so you can access its properties without using the safe call operator:

```
authorName?.let {  
    nameLength = authorName.length  
}
```

You'll learn more about the syntax used to call the `let` function, called *trailing lambda* syntax, in the next section of the book.

Elvis operator

There's another handy way to get a value from a nullable. You use it when you want to get a value out of the nullable *no matter what* — and in the case of `null`, you'll use a default value.

Here's how it works:

```
var nullableInt: Int? = 10  
var mustHaveResult = nullableInt ?: 0
```

The operator used on the second line `?:` is known as the **Elvis operator**, since it resembles a certain rock star when rotated by 90 degrees clockwise.

`?:` $\xrightarrow{\text{rotate}}$ `::~`

Elvis operator

Using the Elvis operator means `mustHaveResult` will equal either the value inside `nullableInt`, or `0` if `nullableInt` contains `null`. In this example, `mustHaveResult` is inferred to be of type `Int` and contains the concrete `Int` value of `10`. The previous code using the Elvis operator is equivalent to the following use of a null check and smart cast, but is more concise:

```
var nullableInt: Int? = 10  
var mustHaveResult = if (nullableInt != null) nullableInt else 0
```

Set the `nullableInt` to `null`, like so:

```
nullableInt = null  
mustHaveResult = nullableInt ?: 0
```

Now `mustHaveResult` equals `0`.

Challenges

You've learned the theory behind nullables and seen them in practice. Now it's your turn to play!

Challenge 1: You be the compiler

Which of the following are valid statements?

```
var name: String? = "Ray"  
var age: Int = null  
val distance: Float = 26.7  
var middleName: String? = null
```

Challenge 2: Divide and conquer

First, create a function that returns the number of times an integer can be divided by another integer without a remainder. The function should return `null` if the division doesn't produce a whole number. Name the function `divideIfWhole`.

Then, write code that tries to extract the nullable result of the function. There should be two cases: upon success, print "Yep, it divides \$answer times", and upon failure, print "Not divisible :[".

Finally, test your function:

1. Divide 10 by 2. This should print "Yep, it divides 5 times"
2. Divide 10 by 3. This should print "Not divisible :["

Hint 1: Use the following as the start of the function signature:

```
fun divideIfWhole(value: Int, divisor: Int)
```

You'll need to add the return type, which will be a nullable!

Hint 2: You can use the modulo operator (%) to determine if a value is divisible by another; recall that this operation returns the remainder from the division of two numbers. For example, `10 % 2 = 0` means that 10 is divisible by 2 with no remainder, whereas `10 % 3 = 1` means that 10 is divisible by 3 with a remainder of 1.

Challenge 3: Refactor and reduce

The code you wrote in the last challenge used `if` statements. In this challenge, refactor that code to use the Elvis operator. This time, make it print "It divides X times" in all cases, but if the division doesn't result in a whole number, then X should be 0.

Key points

- `null` represents the absence of a value.
- Non-null variables and constants must always have a non-null value.
- **Nullable** variables and constants are like boxes that can contain a value *or* be empty (`null`).
- To work with the value inside a nullable, you must typically first check that the value is not null.
- The safest ways to work with a nullable's value is by using **safe calls** or the **Elvis operator**. Use **not-null assertions** only when appropriate, as they could produce a runtime error.

Where to go from here?

Nullables are a core Kotlin feature that helps make the language safe and easy to use. They force you to consciously choose when values should and should not be nullable, and you should prefer the latter as the default case. You'll find yourself using them when needed, making your code safe by ensuring that the absence of values is handled explicitly.

That is something that you'll come to admire over the course of your Kotlin experience! In particular, look forward to using nullables in Section II, where you'll learn about **collections** and **lambdas**.

Section II: Collections & Lambdas

So far, you've mostly seen data in the form of single elements. Although pairs and triples can have multiple pieces of data, you're limited to combining only two or three items with them. In this section, you'll learn about **collection types** in Kotlin. Collections are flexible "containers" that let you store any number of values together.

The chapters in this section will give you the foundation that you need to organize your data and code so that you can begin to do more sophisticated programming in Kotlin!